

ROMPER PARA APRENDER A DEFENDER

Explotar un banco en vivo, ganar cinco niveles de Ethernaut hackeándolos, correr Slither y auditar el contrato de otro equipo. Después, el parcial.

MODALIDAD	REQUISITO	DURACIÓN	META
Ethernaut y auditoría en parejas · parcial individual	MetaMask (curso) en Sepolia + Python 3.10 para Slither	Bloque B ≈ 35 min + trabajo autónomo	5 niveles + informe de auditoría

LA REGLA DE ORO DE HOY

Todo lo de esta sesión ocurre en **redes de prueba (Sepolia) o en la red local**, jamás con dinero real. La billetera es la **del curso**, sin fondos reales. La frase semilla no se comparte con nadie —tampoco con el docente—. Estos contratos son material de clase **sin auditar**: existen para ser rotos y entendidos, nunca para custodiar valor.

01 Qué vas a hacer, y por qué

[LEER PRIMERO](#)

La seguridad de un contrato es distinta de la de una app web: el código es **inmutable** (no hay parche), **público** (cualquiera lo estudia) y **custodia valor** directamente. Para defender, primero hay que saber atacar. Hoy atacas.

PARTE	QUÉ HACES	RESULTADO DE APRENDIZAJE
0 · Reentrancy en vivo	Ves cómo un atacante vacía un banco con la prueba ya escrita.	Entender el ataque de The DAO línea por línea.
1 · Ethernaut	Ganas 5 niveles explotándolos desde la consola del navegador.	RA5 · identificar y explotar vulnerabilidades típicas.
2 · Slither	Corres un analizador estático sobre un contrato y lees su salida.	RA5 · usar herramientas automáticas de análisis.
3 · Auditoría cruzada	Lees el contrato de otro equipo y entregas hallazgos clasificados.	RA5 · remediar; leer código ajeno con criterio.

QUÉ SE ENTREGA (EVIDENCIA)

- **Ethernaut:** una captura de la consola por cada nivel superado (5), y una línea por nivel explicando la vulnerabilidad con tus palabras.
- **Slither:** captura de la salida sobre `BancoVulnerable.sol` y una frase diciendo qué marcó y por qué.
- **Auditoría cruzada #1:** el informe de 1 a 2 páginas con el formato del documento «Auditoría cruzada» del material del proyecto. Vale el 20 % del segundo corte.

El **parcial práctico** (45 % del segundo corte) es aparte, individual, y se explica en la sección 8.

Antes de tocar Ethernaut, mira el ataque completo sobre un contrato de juguete. Todo está en el repositorio de laboratorios. Desde `material/laboratorios-evm` (o tu copia de trabajo):

```
npx hardhat test test/s09/Reentrancy.test.js
```

Cinco pruebas cuentan la historia entera:

CONTRATO	QUÉ DEMUESTRA
BancoVulnerable	Tres clientes honestos depositan 10 ETH. El <code>Atacante</code> deposita 1 y sale con 11 . El banco queda en cero y los saldos honestos quedan impagables.
BancoSeguro	El mismo ataque revierte en cadena y el banco conserva sus 10 ETH. Corrección por CEI + <code>ReentrancyGuard</code> .

Lee `contracts/s09/BancoVulnerable.sol`, `Atacante.sol` y `BancoSeguro.sol` mientras corre. El fallo está en el **orden**: el banco vulnerable envía el ETH *antes* de poner el saldo en cero, así que el `receive()` del atacante vuelve a entrar mientras su saldo sigue en pie.

UN DETALLE FINO QUE VALE LA PENA NOTAR

Contra `BancoSeguro`, el ataque no revierte con `ReentrancyGuardReentrantCall` sino con `EnvioFallido`. La reentrada choca con el guard dentro del `receive()` del atacante, ese `receive()` revierte, y eso hace fallar el envío del banco. La cascada deja todo intacto. El error que llega arriba es el del envío, no el del guard.

Ethernaut, de OpenZeppelin, es un CTF: cada nivel es un contrato con una vulnerabilidad y «ganas» explotándolo. No se programa en una IDE aparte: se juega desde la **consola del navegador**, hablándole al contrato por su ABI.

A

PREPARACIÓN

Billetera, red y fondos

1. Abre <https://ethernaut.openzeppelin.com> y conecta la **MetaMask del curso**.
 2. Selecciona la red **Sepolia** en MetaMask. Ethernaut soporta también Optimism Sepolia, Arbitrum Sepolia, Holesky y Amoy; si eliges una no soportada, el juego te lleva a Sepolia por defecto.
 3. Necesitas algo de ETH de prueba para el gas. Usa el faucet de la unidad o pide al docente que te transfiera desde la wallet institucional.
- Prerrequisito:** el nivel 0 (**Hello Ethernaut**) quedó de trabajo de la Sesión 8. Hoy empiezas en Fallback.

B

EL CICLO DE CADA NIVEL

Instancia, inspección, interacción, envío

1. **Abre la consola.** Herramientas → Herramientas de desarrollo → Console (o F12). Verás mensajes del juego; uno es tu dirección (`player`).
2. **Pide una instancia.** Botón «**Get New Instance**»: Ethernaut despliega un contrato solo para ti y lo expone como la variable `contract`. Autoriza la transacción en MetaMask.
3. **Inspecciona.** Los comandos clave:
 - `player` — tu dirección. · `await getBalance(player)` — tu saldo. · `help()` — utilidades.
 - `contract` — la instancia. · `contract.abi` — **todas las funciones que puedes llamar**. · `await contract.info()` — pistas del nivel.
4. **Interactúa.** `await contract.laFuncion(args)` para llamar; `contract.sendTransaction({value: toWei("0.001")})` para enviar ETH.
5. **Envía.** Botón «**Submit instance**»: Ethernaut comprueba si cumpliste el objetivo.

EL REFLEJO QUE HAY QUE ADQUIRIR

Ante cualquier nivel, **lee primero**: el enunciado, `contract.abi` y el código fuente del nivel (está enlazado en la página, en el ícono de código). La vulnerabilidad casi siempre salta leyendo, antes de tocar nada. Hackear no es adivinar: es entender qué hace el contrato y encontrar la suposición que rompe.

Aquí **no está la solución**: está el método y unas pistas escalonadas. Léelas en orden y detente en cuanto se te encienda la bombilla. Resuélvelos en este orden (deja Force para el final, porque exige desplegar un contrato).

1

FALLBACK • control de acceso

Hazte owner y vacía el contrato

Objetivo: ser `owner` del contrato y dejar su balance en cero.

Pregunta guía: ¿de cuántas formas distintas se puede volver uno `owner` en este contrato? Mira `contract.abi`: ¿qué funciones cambian `owner`?

Pista 1: hay una condición «normal» para ser owner (contribuir más que el dueño, que empieza con 1000 ether) y hay **otra puerta** más barata.

Pista 2: revisa la función `receive()`: ¿qué exige exactamente para cambiar el owner? ¿Cuánto ETH y qué condición previa sobre tu `contribution`?

Pista 3: primero cumple la condición previa con una contribución mínima (menor que `0.001 ether`), y luego dispara al `receive()` con un envío directo pequeño. Ya siendo owner, ¿qué función te deja vaciarlo?

Conecta con: S7 (receive/fallback) y A.8 de hoy (control de acceso mal ubicado).

2

FALLOUT • el constructor que no lo era

Hazte owner

Objetivo: ser `owner`.

Pregunta guía: lee los nombres de las funciones en `contract.abi` con MUCHA atención. ¿Hay alguna que parezca el constructor pero esté escrita como una función pública normal?

Pista 1: este contrato es de una versión vieja de Solidity, cuando el constructor era una función con el **mismo nombre** del contrato. ¿El nombre coincide *exactamente*?

Pista 2: mira el nombre carácter por carácter. ¿Es una «I» o es un «1»? Si no coincide con el nombre del contrato, esa función no es el constructor: es pública y cualquiera puede llamarla.

Pista 3: llámala.

Conecta con: S6 (por qué desde 0.4.22 existe la palabra `constructor`).

5

TOKEN • desbordamiento aritmético

Pasa de 20 a muchísimos tokens

Objetivo: empiezas con 20 tokens; consigue muchos más.

Pregunta guía: el `transfer` hace `require(balances[msg.sender] - _value >= 0)`. Con enteros **sin signo** (`uint`), ¿ese `require` puede ser alguna vez falso?

Pista 1: una resta de `uint` nunca es negativa. Si restas más de lo que tienes, el número **da la vuelta** (underflow) hasta un valor gigantesco. Este contrato corre en Solidity 0.6, sin la protección de 0.8.

Pista 2: ¿qué pasa con `balances[msg.sender]` si transfieres **más de 20** a cualquier otra dirección? El chequeo pasa, y tu saldo desborda.

Pista 3: transfiere 21 (o más) a una dirección cualquiera distinta de la tuya. Verifica con `(await contract.balanceOf(player)).toString()`.

Conecta con: A.10 de hoy y el caso BeautyChain (CVE-2018-10299, abr 2018).

Vacía el balance del contrato

Objetivo: dejar el balance de la instancia en cero. Es el mismo fallo de la Parte 0, pero lo explotas tú.

Pregunta guía: en `withdraw`, ¿en qué orden ocurren el envío de ETH y el descuento del saldo? ¿Qué se ejecuta en tu contrato cuando recibes ese ETH?

Pista 1: el contrato envía con `call{value:...}` antes de restar tu saldo. Necesitas un **contrato atacante** con un `receive()` que vuelva a llamar a `withdraw`.

Pista 2: primero deposita (con `donate`) una cantidad que puedas retirar; consulta el balance de la instancia con `await getBalance(instance)` para no pedir más de lo que hay.

Pista 3: en el `receive()` de tu atacante, si aún queda balance en la instancia, vuelve a llamar `withdraw`. Desplégalo en Remix (Sepolia), pásale la dirección de la instancia, y dispara el ataque. La estructura es idéntica al `Atacante.sol` de la Parte 0.

Conecta con: A.3 a A.7 de hoy y The DAO (jun 2016, ~3,6 M ETH).

Deja el balance del contrato en positivo

Objetivo: hacer que el balance del contrato sea mayor que cero. Pero `Force` no tiene `receive`, ni `fallback`, ni funciones `payable`: no acepta ETH por las vías normales.

Pregunta guía: ¿existe alguna forma de **forzar** ETH hacia un contrato que no quiere recibirlo? ¿Qué le pasa al saldo de un contrato cuando otro se autodestruye apuntándole?

Pista 1: `selfdestruct(objetivo)` envía todo el saldo del contrato que se destruye a `objetivo`, aunque `objetivo` no tenga forma de recibirlo.

Pista 2: despliega en Remix un contrato pequeño con un `constructor() payable {}`, mándale 1 wei al desplegarlo, y dale una función que haga `selfdestruct(payable(<dirección de la instancia>))`.

Nota de actualidad: tras Dencun (EIP-6780, mar 2024) `selfdestruct` ya no borra el contrato salvo en la misma transacción de creación, pero **sigue transfiriendo el saldo**: el patrón «constructor payable + selfdestruct» sigue resolviendo el nivel.

Conecta con: la lección de que `address(this).balance == 0` nunca es una invariante segura.

TIEMPOS SUGERIDOS (BLOQUE B, ~35 min de Ethernaut)

Fallback 5 min · Fal1out 3 min · Token 7 min · Re-entrancy 10-15 min (el más largo, exige desplegar) · Force 8-10 min. Lo que no alcances queda de trabajo autónomo (pregunta de la semana).

Slither revisa el código *sin ejecutarlo* y marca patrones peligrosos en segundos. Es el primer filtro de seguridad, no el último: no entiende la lógica de negocio ni encuentra fallos de diseño nuevos.

INSTALACIÓN (REQUIERE PYTHON 3.10 O SUPERIOR)

Comprueba primero que tienes Python: `python --version` (o `python3 --version`). Luego:

```
python -m pip install slither-analyzer
```

Slither necesita el compilador de Solidity en la versión del contrato. La forma limpia de manejarlo es `solc-select`:

```
python -m pip install solc-select
solc-select install 0.8.34
solc-select use 0.8.34
```

Si tu sala no permite instalar Python, hay una **extensión de VS Code** de Slither y alternativas web; úsalas y dilo en tu evidencia.

Corre Slither sobre el banco vulnerable (desde `material/laboratorios-evm` o tu copia):

```
slither contracts/s09/BancoVulnerable.sol
```

Esta es la **salida real** (se ejecutó con Slither 0.11.6 + solc 0.8.34):

```
INFO:Detectors:
Reentrancy in BancoVulnerable.retirar() (contracts/s09/BancoVulnerable.sol#21-30):
  External calls:
    - (ok,None) = address(msg.sender).call{value: monto}() (#26)
  State variables written after the call(s):
    - saldos[msg.sender] = 0 (#29)
  BancoVulnerable.saldos can be used in cross function reentrancies
Reference: ../Detector-Documentation#reentrancy-vulnerabilities-1
INFO:Detectors:
Low level call in BancoVulnerable.retirar() (#26)
Reference: ../Detector-Documentation#low-level-calls
INFO:Slither: analyzed (1 contracts with 102 detectors), 2 result(s) found
```

HALLAZGO

CÓMO LEERLO

reentrancy-eth (ALTA)

El de arriba. Dice exactamente lo mismo que vimos: hay una llamada externa (línea 26) y **después** se escribe el estado (línea 29). Es el fallo. **Actúa sobre él.**

low-level-calls (Informativa)

Marca el uso de `call{value:}`. No es un bug: es un aviso de que hay una llamada de bajo nivel. Con el patrón de retiro es correcto. **Falso positivo en este contexto.**

LA LECCIÓN: SEPARAR SEÑAL DE RUIDO

Corre Slither ahora sobre `contracts/s09/BancoSeguro.sol` (usa `--filter-paths node_modules` para ignorar el ruido de OpenZeppelin). Verás que el hallazgo **ALTO desaparece**: solo queda el aviso informativo del `call`. Esa diferencia —de dos hallazgos a uno informativo— es la prueba de que la corrección funcionó. Saber cuáles hallazgos son reales y cuáles son ruido es justo lo que se evalúa en la auditoría cruzada.

Cada equipo entrega el contrato principal de su proyecto y recibe el de otro equipo. Media hora para auditarlo con una lista concreta. **Leer código ajeno es parte de la competencia profesional.**

1. **Recorre la taxonomía** de las ocho vulnerabilidades (A.2 del deck), una por una, contra el contrato recibido. Pregúntate por cada una: ¿aplica aquí? ¿dónde?
2. **Pasa Slither** (o la extensión de VS Code). Separa hallazgos reales de falsos positivos, como hiciste en la sección 5.
3. **Clasifica cada hallazgo** con severidad (crítica / alta / media / baja / informativa) y la **línea exacta**.
4. **Propón la corrección mínima** de cada uno. **No reescribas** el contrato ajeno.

FORMATO, EQUIPOS Y RÚBRICA

El **formato de cada hallazgo** (título, severidad con criterios, ubicación, descripción, escenario de explotación y recomendación), **cómo se asignan los equipos**, qué se entrega y la **rúbrica** están en el documento «Auditoría cruzada» (HTML + PDF) del material del proyecto. Se califica la **calidad de la auditoría**, no cuántos fallos tenga el contrato revisado: un informe que dice «no hallé vulnerabilidades explotables; revisé esto y esto» vale.

07 Errores frecuentes

CASI TODO SE RESUELVE AQUÍ

SÍNTOMA	CAUSA Y SOLUCIÓN
Ethernaut no reconoce <code>contract</code>	No pediste instancia todavía, o la transacción de «Get New Instance» aún no confirmó. Espera unos segundos y vuelve a intentar.
«insufficient funds» al enviar	No tienes ETH de prueba en Sepolia para el gas. Usa el faucet de la unidad o pide al docente.
Una llamada «pending» que no resuelve en consola	Expande la promesa o usa <code>await</code> delante (p. ej. <code>await contract.info()</code>).
El exploit de Re-entrancy no vacía todo	Depositaste/donaste menos que el balance de la instancia, o tu <code>receive()</code> no vuelve a llamar mientras quede balance. Revisa la condición.
Force: «no puedo enviarle ETH»	Exacto: por eso hay que forzarlo con <code>selfdestruct</code> desde otro contrato, no con un envío normal.
Slither: «solc not found» o versión equivocada	Instala y fija la versión con <code>solc-select install 0.8.34</code> y <code>solc-select use 0.8.34</code> .
Slither marca decenas de cosas de OpenZeppelin	Es ruido de las dependencias. Filtra con <code>--filter-paths node_modules</code> .

08 Parcial práctico

INDIVIDUAL · 90 MIN · 45 % DEL CORTE

La evaluación calificada de la sesión va aparte y es **individual**. Se entrega un contrato vulnerable al empezar el bloque C. Tres tareas:

1. **Identificar** cada fallo: nombre, severidad y línea exacta.
2. **Explotar** al menos uno con una prueba de Hardhat.
3. **Corregir** los fallos, sin romper lo legítimo y **sin reescribir** el contrato entero.

CONDICIONES

90 minutos exactos, en máquina, en el repositorio del curso. **Libro abierto**: material, documentación oficial y el repositorio. **Sin asistentes de IA** (Copilot, ChatGPT y similares) ni ayuda de otra persona: el parcial mide TU criterio de seguridad, que es justo lo que un asistente taparía. Una sospecha fundada anula el parcial. Se entrega el contrato corregido, la prueba del exploit y media página con los hallazgos y sus correcciones. La rúbrica es sobre 20 puntos.

Sugerencia de reparto: ~25 min leer e identificar, ~35 min escribir el exploit, ~25 min corregir, ~5 min redactar la media página.

09 Preguntas de reflexión

MIRA EL ARTEFACTO REAL

1. Abre `contracts/s09/BancoVulnerable.sol` y `BancoSeguro.sol` en dos ventanas. Señala la **única diferencia de orden** que convierte a uno en robo y al otro en seguro. ¿Cuánto gas extra cuesta la corrección por CEI? (Pista: cero.)
2. De los cinco niveles que resolviste, ¿cuál se parece más a un fallo que podría tener **tu proyecto**? Justifícalo mirando tu propio contrato, no en abstracto.
3. Slither marcó `low-level-calls` también en `BancoSeguro`, que es correcto. Explica por qué un hallazgo automático puede ser cierto y aun así **no ser una vulnerabilidad**. ¿Qué te dice eso sobre confiar ciegamente en la herramienta?

10 Lista de chequeo y referencias

ANTES DE ENTREGAR

OK

Los 5 niveles de Ethernaut superados, con captura de cada uno.

Una línea por nivel explicando la vulnerabilidad con tus palabras.

Captura de la salida de Slither sobre BancoVulnerable + una frase de lectura.

Informe de auditoría cruzada #1 con el formato del material del proyecto.

Las tres preguntas de reflexión respondidas.

- Ethernaut · <https://ethernaut.openzeppelin.com>
- Slither · <https://github.com/crytic/slither> · solc-select · <https://github.com/crytic/solc-select>
- The DAO (jun 2016), Parity (jul y nov 2017), BeautyChain (CVE-2018-10299), Mango Markets (oct 2022): casos citados en el deck de la sesión, con fuentes verificadas.